

Приложение А

Листинг программы

```
// ===== CLASS SAVEMANAGER =====

class SaveManager {
private:
    string filename = "save.dat"; // Имя файла сохранения

    int maxUnlockedLevel; // Максимально открытый уровень

    int currentLevel; // Текущий выбранный уровень

public:
    // При запуске загружается сохранение
    SaveManager() : maxUnlockedLevel(1), currentLevel(1) {
        load();
    }

    // Загрузка прогресса из файла
    void load() {
        ifstream file(filename, ios::binary);

        if (file) {
            file.read(reinterpret_cast<char*>(&maxUnlockedLevel),
                sizeof(maxUnlockedLevel));

            file.read(reinterpret_cast<char*>(&currentLevel), sizeof(currentLevel));
        }
        else {
            maxUnlockedLevel = 1;

            currentLevel = 1;

            save(); // если файла нет, создаём дефолтное сохранение
        }

        // Сохраняет текущий прогресс
        void save() {
            ofstream file(filename, ios::binary);

            if (file) {
                file.write(reinterpret_cast<char*>(&maxUnlockedLevel),
                    sizeof(maxUnlockedLevel));

                file.write(reinterpret_cast<char*>(&currentLevel),
                    sizeof(currentLevel));
            }

            int getMaxUnlockedLevel() const { return maxUnlockedLevel; }

            int getCurrentLevel() const { return currentLevel; }

            void setCurrentLevel(int level) {
                if (level != currentLevel) {
                    currentLevel = level;

                    save();
                }
            }

            void unlockLevel(int level) {
                if (level > maxUnlockedLevel) {
                    maxUnlockedLevel = level;

                    save();
                }
            }

            // Полный сброс прогресса
            void resetProgress() {
                maxUnlockedLevel = 1;

                currentLevel = 1;

                save();
            }

            // ===== CLASS CLIENT =====

            class Client {
            public:
                //Инициализация

                // Конструктор клиента с заказом и его характеристиками
                Client(const string& F, float X, float Y, float patienceSeconds,

                    const vector<string>& clientOrders, bool sausage, bool cheese, bool pepper,

                    bool chocolate, bool banana, bool cake1, bool cake2, const wstring&
                    clientName)

```

```

                : file(F), orders(clientOrders), requireSausage(sausage),
                requireCheese(cheese),

                requirePepper(pepper), requireChocolate(chocolate), requireBanana(banana),

                requireCake1(cake1), requireCake2(cake2), x(322), y(59),

                currentFrame(0), animationAccumulatedTime(0.0f),

                state(Growing), patienceTime(patienceSeconds), isGone(false),

                name(clientName), elapsedPatience(0.0f)

            {
                servedOrders.clear();

                if (!texture.loadFromFile("Characters/" + F)) {
                    cout << "Ошибка загрузки персонажа: Characters/" << F << endl;

                    // Добавление блюда в список отданных, если заказано

                    bool serveDish(const string& dishType) {

                        int orderedCount = count(orders.begin(), orders.end(), dishType);

                        int servedCount = count(servedOrders.begin(), servedOrders.end(),
                            dishType);

                        if (servedCount < orderedCount) {
                            servedOrders.push_back(dishType);

                            return true;
                        }

                        return false;
                    }

                    // Проверка, все ли блюда выданы
                    bool allOrdersServed() const {
                        return servedOrders.size() == orders.size();
                    }

                    // Обновление таймера терпения и анимации
                    void update(float dt, bool isGamePaused) {
                        if (!isGamePaused) {
                            updateAnimation(dt);

                            if (state == Waiting && !isGamePaused) {
                                elapsedPatience += dt;

                                if (elapsedPatience >= patienceTime) {
                                    state = Leaving;

                                    currentFrame = frameCount - 1;
                                }
                            }
                            else if (state == Leaving && !isGamePaused) {
                                animationAccumulatedTime += dt;

                                if (animationAccumulatedTime >= frameDuration) {
                                    animationAccumulatedTime -= frameDuration;

                                    currentFrame--;

                                    if (currentFrame < 0) {
                                        currentFrame = 0;

                                        isGone = true;
                                    }

                                    IntRect frameRect(currentFrame * 294, 0, 294, 237);

                                    sprite.setTextureRect(frameRect);

                                    ...
                                }

                                // Переход в режим благодарности
                                void completeOrder() {
                                    if (state == Waiting) {
                                        state = StoryTelling;

                                        currentFrame = frameCount - 1;
                                    }

                                    // Если заказ не принят — клиент уходит
                                    void failOrder() {
                                        if (state == Waiting) {
                                            state = Leaving;
                                        }
                                    }
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

        currentFrame = frameCount - 1;}}

bool isReadyForOrder() const {

    return state == Waiting;}};

// ===== CLASS MACHINE =====

class Machine {

public:

//Инициализация

    // Конструктор: принимает пути к изображениям, позицию и длительность
    приготовления

    Machine(string idleImg, string preparingImg, string finishedImg, Vector2f pos,

        float t, string type, int animFrames = 1)

        : prepTime(t), state(Idle), machineType(type), preparationProgress(0.0f),

        animationFrames(animFrames), currentAnimationFrame(0),

        animationDuration(0.1f), animationElapsedTime(0.0f)

    {

        // Загрузка всех фаз машины

// Запускает приготовление

void start() {

    if (state == Idle) {

        state = Preparing;

        preparationProgress = 0.0f;

        sprite.setTexture(texturePreparing);

        currentAnimationFrame = 0;

        animationElapsedTime = 0.0f;}}

// Обновление состояния машины

void update(float dt, bool isGamePaused) {

    if (state == Preparing && !isGamePaused) {

        preparationProgress += dt / prepTime;

        // Анимация в процессе приготовления

        animationElapsedTime += dt;

        if (animationElapsedTime >= animationDuration) {

            animationElapsedTime = 0.0f;

            currentAnimationFrame = (currentAnimationFrame + 1) %

animationFrames;

            animationRect.left = currentAnimationFrame * 277.8975f;

            sprite.setTextureRect(animationRect);}

        // Завершение

        if (preparationProgress >= 1.0f) {

            state = Finished;

            sprite.setTexture(textureFinished);

            sprite.setTextureRect(IntRect(0, 0, textureFinished.getSize().x,

textureFinished.getSize().y));

            preparationProgress = 1.0f;

...}

// Отрисовка машины и прогресс-бара

// Проверяет, был ли клик по машине

bool isClicked(Vector2f mousePos) {

    return sprite.getGlobalBounds().contains(mousePos);}

// Сброс машины в начальное состояние

void reset() {

    state = Idle;

    preparationProgress = 0.0f;

```

```

        sprite.setTexture(textureIdle);

        if (animationFrames > 1) {

            sprite.setTextureRect(IntRect(0, 0, textureIdle.getSize().x,

textureIdle.getSize().y));

...};

// ===== CLASS GAME =====

class Game {

private:

    // Состояния игры

    enum GameState { LOADING, MAIN_MENU, LEVEL_SELECT,

GAME_PLAY, FINAL_SCREEN, LEVEL_RESULTS } state;

    Client* currentClient;    // Текущий клиент на экране

    int clientsServed;        // Сколько обслужено за уровень

    int successfulServed;     // Сколько успешно

    // Истории персонажей по уровням

    map<int, map<string, vector<wstring>>>> levelStories;

    Machine* coffeeMachine;   // Кофейная машина

    Clock deltaClock;         // Таймер обновлений

    vector<string> currentOrders; // Текущий заказ

    Music backgroundMusic;    // Фоновая музыка

    bool soundOn;             // Флаг звука

    SaveManager saveManager;  // Сохраняет уровень и прогресс

    int currentLevelNumber;    // Активный уровень

    // Графика уровня

    Texture levelBgTexture;

    Sprite levelBgSprite;

    Texture counterTexture;

    Sprite counterSprite;

    OrderWindow orderWindow;  // Окно заказа

    DialogCloud dialogCloud;  // Облако диалога

    enum OrderState { WAITING, PREPARING, READY_TO_SERVE }

orderState;

    bool orderCorrect;

    float coffeePreparationTime; // Время готовки кофе

    // Блюда

    CroissantDish* croissantDish;

    SandwichDish* sandwichDish;

    // Для перетаскивания

    CroissantDish* currentDragCroissant;

    SandwichDish* currentDragSandwich;

    DraggableItem* currentDragItem;

    vector<string> availableCharacters; // Очередь персонажей

    Clock clientSpawnTimer;

    bool spawningClient;

    // Области для объектов

    // Сброс текущего уровня: очищает ингредиенты, сбрасывает машины и

диалог

    void resetLevel() {

        activeIngredients.clear();    // Удаляем все активные (перетаскиваемые)

ингредиенты

        availableCharacters.clear();  // Очищаем очередь доступных клиентов

        isFinalLevelCompleted = false; // Сброс флага при новом уровне

        // Удаляем круассан, если он есть

```

```

    if (croissantDish) {

        delete croissantDish;

        croissantDish = nullptr;

    // Удаляем сэндвич, если он есть

    if (sandwichDish) {

        delete sandwichDish;

        sandwichDish = nullptr;

    }

    coffeeMachine->reset();      // Сбрасываем кофемашину в начальное
    состояние

    // Убираем указатели на объекты, которые могли перетаскиваться

    orderState = WAITING;      // Состояние заказа — ждём нового клиента

    orderWindow.visible = false;  // Скрываем окно заказа

    clientsServed = 0;          // Счётчики для текущего уровня

    successfulServed = 0;

    clientTimes.clear();        // Обнуляем список времен обслуживания

    gameSoundButton.forceUpdate(); // Обновляем кнопку звука (на случай
    возврата)

public:

    Game()

        // Создаём главное окно размером 1280x720 с заголовком и
        ограниченными стилями

        : window(VideoMode(1280, 720), L"Кафе Мечта", Style::Titlebar |
        Style::Close),

        // Инициализация начального состояния игры

        state(LOADING),

        soundOn(true),

        saveManager(),

        // Передаём ссылки на менеджер сохранения и флаг звука в главное меню

        mainMenu(&saveManager, &soundOn),

        // Пока меню уровней не создано

        levelSelectMenu(nullptr),

        // Кнопка звука в игровом интерфейсе

        gameSoundButton(&soundOn),

        // Инициализация указателей и счётчиков

        currentClient(nullptr),

        clientsServed(0),

        successfulServed(0),

        // Загружаем текущий уровень из сохранения

        currentLevelNumber(saveManager.getCurrentLevel()),

        // Состояние заказа и кофе-машины

        orderState(WAITING),

        orderCorrect(false),

        coffeePreparationTime(5.0f),

        // Указатели на блюда

        croissantDish(nullptr),

        sandwichDish(nullptr),

        currentDragCroissant(nullptr),

        currentDragSandwich(nullptr),

        currentDragItem(nullptr),

        // Генерация клиентов и координаты игровых зон

        spawningClient(false),

        // Финальный экран и таймер уровня

```

```

        finalButtonHovered(false),

        levelElapsedTime(0.0f),

        currentClientSpawnTime(0.0f),

        // Состояние паузы и кнопок в паузе

        gamePaused(false),

        pauseButtonHovered(false),

        playButtonHovered(false),

        mainMenuButtonHovered(false),

        isFinalLevelCompleted(false)

    { ...

        // Инициализация кофе-машины с 4 кадрами анимации

        coffeeMachine = new Machine(

            "coffee_idle.png",

            "coffee_preparing.png",

            "coffee_finished.png",

            Vector2f(39.81f, 138.71f),

            5.0f,

            "coffee",

            4

        );

        // Загрузка всех диалогов персонажей по уровням

        loadStories();

        // Освобождаем ресурсы при завершении игры

        ~Game() {

            if (levelSelectMenu) delete levelSelectMenu;  // Удаляем меню выбора
            уровня

            if (currentClient) delete currentClient;      // Освобождаем текущего
            клиента

            delete coffeeMachine;                          // Удаляем кофе-машину

            if (croissantDish) delete croissantDish;      // Удаляем текущий круассан

            if (sandwichDish) delete sandwichDish;        // Удаляем текущий сэндвич

        }

        void loadStories() {

            levelStories.clear();

            levelStories[1]["character1"] = {

                L"12 издательств сказали 'нет'... Но Гарри заслуживает жизнь.",

                L"Буду писать здесь каждый день. Хотя бы строчку."};

            levelStories[1]["character2"] = {

                L"Украли моего Освальда... Но я придумал лучше!",

                L"Микки... да, он заговорит! Все назовут безумцем? Пусть!";

            }

            levelStories[1]["character3"] = {

                L"Falcon 1 взорвался. Третий раз. Денег почти нет.",

                L"Четвертый запуск будет. Марс не покорится без жертв."};

            ...      // Если предыдущий клиент ещё существует — удаляем его

            if (currentClient) delete currentClient;

            // Если список доступных персонажей пуст — создаём новый набор и
            перемешиваем его

            if (availableCharacters.empty()) {

                availableCharacters = { "character1.png", "character2.png",
                "character3.png" };

                random_shuffle(availableCharacters.begin(), availableCharacters.end()); //
                случайный порядок появления

                // Получаем имя файла текстуры нового клиента (последнего в
                перемешанном списке)

                string characterFile = availableCharacters.back();

```



```

if (event.type == Event::MouseMove) {
    mousePos = Vector2f(static_cast<float>(event.mouseMove.x),
        static_cast<float>(event.mouseMove.y));
}
else if (event.type == Event::MouseButtonPressed ||
    event.type == Event::MouseButtonReleased) {
    mousePos = Vector2f(static_cast<float>(event.mouseButton.x),
        static_cast<float>(event.mouseButton.y));
}
if (state == MAIN_MENU) {
    // Наведение на кнопки
    if (event.type == Event::MouseMove) {
        mainMenu.update(mousePos.x, mousePos.y);
    }
    // Клик по кнопкам
    if (event.type == Event::MouseButtonPressed &&
        event.mouseButton.button == Mouse::Left) {
        int action = mainMenu.handleClick(mousePos.x, mousePos.y);
        if (action == 0) {
            // Новая игра
            saveManager.resetProgress();
            currentLevelNumber = 1;
            state = LEVEL_SELECT;
            if (levelSelectMenu) delete levelSelectMenu;
            levelSelectMenu = new LevelSelectMenu(&saveManager,
&soundOn);
            else if (action == 1) {
                // Продолжить игру
                state = LEVEL_SELECT;
                if (levelSelectMenu) delete levelSelectMenu;
                levelSelectMenu = new LevelSelectMenu(&saveManager,
&soundOn);
            }
            else if (action == 2) {
                // Выход из игры (сохраняем текущий уровень)
                saveManager.setCurrentLevel(currentLevelNumber + 1);
                window.close();
            }
            // Esc — закрыть окно «о программе», если открыто
            if (event.type == Event::KeyPressed &&
                event.key.code == Keyboard::Escape) {
                if (mainMenu.aboutVisible) mainMenu.aboutVisible = false;
                helpMenuHovered = helpMenuBounds.contains(mousePos);
                helpMenuText.setFillColor(helpMenuHovered ? Color(200, 200, 255) :
Color::White);
            }
            else if (state == LEVEL_SELECT && levelSelectMenu) {
                // Наведение мыши — обновление иконок уровней
                if (event.type == Event::MouseMove) {
                    levelSelectMenu->update(mousePos.x, mousePos.y);
                }
                // Клик по кнопкам
                if (event.type == Event::MouseButtonPressed &&
                    event.mouseButton.button == Mouse::Left) {
                    int action = levelSelectMenu->handleClick(mousePos.x,
mousePos.y);
                    if (action == 0) {
                        // Кнопка "Назад" — возвращаемся в главное меню
                        state = MAIN_MENU;

```

```

mainMenu.reload();
}
else if (action > 0) {
    // Выбор уровня
    currentLevelNumber = action;
    saveManager.setCurrentLevel(currentLevelNumber);
    state = GAME_PLAY;
    resetLevel();    // обрасываем состояние игры
    levelElapsedTime = 0.0f;
    spawnClient();    // запускаем первого клиента
}
// Esc — возвращение в главное меню
if (event.type == Event::KeyPressed &&
    event.key.code == Keyboard::Escape) {
    state = MAIN_MENU;
    mainMenu.reload();
}
else if (state == GAME_PLAY) {
    if (event.type == Event::MouseMove) {
        Vector2f mousePos =
static_cast<Vector2f>(Mouse::getPosition(window));
        if (!gamePaused) {
            pauseButtonHovered =
pauseButtonSprite.getGlobalBounds().contains(mousePos);
            pauseButtonSprite.setTexture(pauseButtonHovered
? pauseButtonHoverTex : pauseButtonNormalTex);
        }
        else {
            playButtonHovered = playButtonBounds.contains(mousePos);
            playButtonSprite.setTexture(playButtonHovered
? playButtonHoverTex : playButtonNormalTex);
            mainMenuButtonHovered =
mainMenuButtonBounds.contains(mousePos);
            mainMenuButtonSprite.setTexture(mainMenuButtonHovered
? mainMenuButtonHoverTex : mainMenuButtonNormalTex);
        }
        if (event.type == Event::MouseButtonPressed &&
            event.mouseButton.button == Mouse::Left) {
            Vector2f mousePos =
static_cast<Vector2f>(Mouse::getPosition(window));
            // Нажатие на кнопку паузы
            if (!gamePaused &&
                pauseButtonSprite.getGlobalBounds().contains(mousePos)) {
                gamePaused = true;
                if (soundOn) {
                    clickSound.play();
                }
                continue;
            }
            // Обработка кнопок в меню паузы
            else if (gamePaused) {
                if (playButtonBounds.contains(mousePos)) {
                    gamePaused = false;
                    if (soundOn) {
                        clickSound.play();
                    }
                }
                else if (mainMenuButtonBounds.contains(mousePos)) {
                    saveManager.setCurrentLevel(currentLevelNumber);
                    state = MAIN_MENU;
                    resetLevel();
                    gamePaused = false;
                    if (soundOn) {
                        clickSound.play();
                    }

```



```

        else if (sandwichDish->hasPepper)

            dishType = "sandwich_pepper";

        else

            dishType = "sandwich_plain";

        bool served = currentClient->serveDish(dishType);

        if (served) {

            orderWindow.setServedOrders(currentClient-
>servedOrders);

            delete sandwichDish;

            sandwichDish = nullptr;

            serveOrder();

        }

        else {

            currentClient->failOrder();

            currentDragSandwich->resetPosition();

        }

        else {

            currentDragSandwich->resetPosition();

            currentDragSandwich->stopDrag();

            currentDragSandwich = nullptr;

        }

        ...

    }

    else if (event.type == Event::MouseMove) {

        if (!gamePaused) {

            if (currentDragItem) {

                currentDragItem->drag(mousePos);

            }

            if (currentDragCroissant) {

                currentDragCroissant->drag(mousePos);

            }

            if (currentDragSandwich) {

                currentDragSandwich->drag(mousePos);

            }

            gameSoundButton.update(mousePos);

        }

        else if (event.type == Event::KeyPressed && event.key.code ==
Keyboard::Escape) {

            gamePaused = !gamePaused;

            if (soundOn && gamePaused) {

                clickSound.play();

            }

        }

        ...

    }

    else if (state == FINAL_SCREEN) {

        if (event.type == Event::MouseButtonPressed) {

            // Любый клик возвращает в главное меню

            if (soundOn) {

                clickSound.play();

            }

            state = MAIN_MENU;

            mainMenu.reload();

        }

        else if (state == LEVEL_RESULTS) {

            if (event.type == Event::MouseMove) {

                Vector2f mousePos(static_cast<float>(event.mouseMove.x),

                static_cast<float>(event.mouseMove.y));

                levelResultsScreen.update(mousePos);

            }

            else if (event.type == Event::MouseButtonPressed &&
event.mouseButton.button == Mouse::Left) {

                Vector2f mousePos(static_cast<float>(event.mouseButton.x),

                static_cast<float>(event.mouseButton.y));

                if (levelResultsScreen.buttonBounds.contains(mousePos)) {

                    if (soundOn) {

```

```

                        clickSound.play();

                        // Если это был финальный уровень - переходим к финальному
экрану

                    }

                }

            }

            if (isFinalLevelCompleted) {

                state = FINAL_SCREEN;

            }

            else {

                state = LEVEL_SELECT;

                if (levelSelectMenu) delete levelSelectMenu;

                levelSelectMenu = new LevelSelectMenu(&saveManager,
&soundOn);

                ...

            }

            else if (event.type == Event::KeyPressed && event.key.code ==
Keyboard::Escape) {

                state = LEVEL_SELECT;

                if (levelSelectMenu) delete levelSelectMenu;

                levelSelectMenu = new LevelSelectMenu(&saveManager,
&soundOn);

                ...

            }

        }

        void update(float dt) {

            backgroundMusic.setVolume(soundOn ? 50 : 0);

            if (state == GAME_PLAY && !gamePaused) {

                gameSoundButton.updateState();

                levelElapsedTime += dt;

                if (currentClient) {

                    currentClient->update(dt, gamePaused);

                }

                if (dialogCloud.completed && currentClient && currentClient->state ==
Client::StoryTelling) {

                    currentClient->state = Client::Leaving;

                    currentClient->currentFrame = currentClient->frameCount - 1;

                }

                if (currentClient && currentClient->state == Client::Waiting) {

                    orderWindow.visible = true;

                }

                else if (currentClient && currentClient->isGone) {

                    float clientTime = levelElapsedTime - currentClientSpawnTime;

                    clientTimes.push_back(clientTime);

                    delete currentClient;

                    currentClient = nullptr;

                    clientsServed++;

                }

                if (clientsServed >= 3) {

                    float totalTime = levelElapsedTime;

                    float avgTime = 0.0f;

                    if (!clientTimes.empty()) {

                        for (float t : clientTimes)

                            avgTime += t;

                        avgTime /= clientTimes.size();

                    }

                    levelResultsScreen.setResults(

                        currentLevelNumber,

                        successfulServed,

                        3,

                        totalTime

                    );

                    // Всегда показываем экран результатов

                    state = LEVEL_RESULTS;

                    // Разблокируем следующий уровень, если это не 6 уровень

```

```

        if (successfulServed >= 1 && currentLevelNumber < 6) {

            saveManager.unlockLevel(currentLevelNumber + 1);}

        // Для 6 уровня переходим сразу к финалу после результатов

        if (currentLevelNumber == 6) {

            // Запомним что это финальный уровень

            isFinalLevelCompleted = true;}}

    else {

        spawningClient = true;

        clientSpawnTimer.restart();

        orderWindow.visible = false;

        dialogCloud.visible = false;

        dialogCloud.completed = true;}

    else if (spawningClient) {

        if (clientSpawnTimer.getElapsedTime().asSeconds() > 2.0f) {

            spawnClient();

            spawningClient = false;}}

    coffeeMachine->update(dt, gamePaused);

    dialogCloud.update(dt);

    dialogCloud.textOffset = Vector2f(120, 50);

    dialogCloud.maxTextWidth = 350;}}

void draw() {

    // Очищаем окно чёрным цветом перед новой отрисовкой

    window.clear(Color::Black);

    // Заставка загрузки

    if (state == LOADING) {

        loadingScreen.draw(window);}

    // Главное меню

    else if (state == MAIN_MENU) {

        mainMenu.draw(window);

        window.draw(menuBar);

        window.draw(helpMenuText); // Добавьте эту строку}

    // Меню выбора уровня

    else if (state == LEVEL_SELECT && levelSelectMenu) {

        levelSelectMenu->draw(window);}

    // Игровой процесс

    else if (state == GAME_PLAY) {

        // Отрисовка фона уровня и прилавка

        window.draw(levelBgSprite);

        window.draw(counterSprite);

        // Отрисовка активных блюд

        if (croissantDish) window.draw(croissantDish->sprite);

        if (sandwichDish) window.draw(sandwichDish->sprite);

        // Отрисовка всех активных ингредиентов на экране

        for (auto& ingredient : activeIngredients) {

            window.draw(ingredient.sprite);}

        // Отрисовка кофе-машины и прогресса приготовления

        coffeeMachine->draw(window);

        // Отрисовка клиента, если он есть

        if (currentClient) {

            window.draw(currentClient->sprite);

            // Имя клиента над его головой

```

```

            Text nameText;

            nameText.setFont(font);

            nameText.setString(currentClient->name);

            nameText.setCharacterSize(24);

            nameText.setFillColor(Color(139, 69, 19)); // коричневый цвет

            nameText.setOutlineColor(Color::White); // белая обводка

            nameText.setOutlineThickness(2);

            FloatRect clientBounds = currentClient->sprite.getGlobalBounds();

            nameText.setPosition(

                clientBounds.left + clientBounds.width / 2 -
                nameText.getLocalBounds().width / 2,

                clientBounds.top - 20

            );

            window.draw(nameText);}

    // Текстовая информация: номер уровня и сколько клиентов обслужено

    Text infoText;

    infoText.setFont(font);

    infoText.setString(L"Уровень: " + to_wstring(currentLevelNumber) +

        L"\nКлиентов: " + to_wstring(clientsServed) + L"/3");

    infoText.setCharacterSize(24);

    infoText.setFillColor(Color(139, 69, 19));

    infoText.setOutlineColor(Color::White);

    infoText.setOutlineThickness(2);

    infoText.setPosition(50, 50);

    window.draw(infoText);

    // Таймер ожидания клиента

    if (currentClient && currentClient->state == Client::Waiting) {

        float remainingTime = currentClient->patienceTime - currentClient-
        >elapsedPatience;

        if (remainingTime < 0) remainingTime = 0;

        Text timerText;

        timerText.setFont(font);

        timerText.setString(L"Время: " +

            to_wstring(static_cast<int>(remainingTime)));

        timerText.setCharacterSize(24);

        timerText.setFillColor(Color(139, 69, 19));

        timerText.setOutlineColor(Color::White);

        timerText.setOutlineThickness(2);

        timerText.setOrigin(timerText.getLocalBounds().width / 2.f, 0.f);

        timerText.setPosition(640, 10); // центр верхней части экрана

        window.draw(timerText);}

    // Отображение окна с заказом клиента

    orderWindow.draw(window);

    // Рисуем перетаскиваемые объекты поверх всего

    if (currentDragItem) {

        window.draw(currentDragItem->sprite);}

    if (currentDragCroissant) {

        window.draw(currentDragCroissant->sprite);}

    if (currentDragSandwich) {

        window.draw(currentDragSandwich->sprite);}

    // Отображаем всё, что нарисовано

    window.display();}};

```